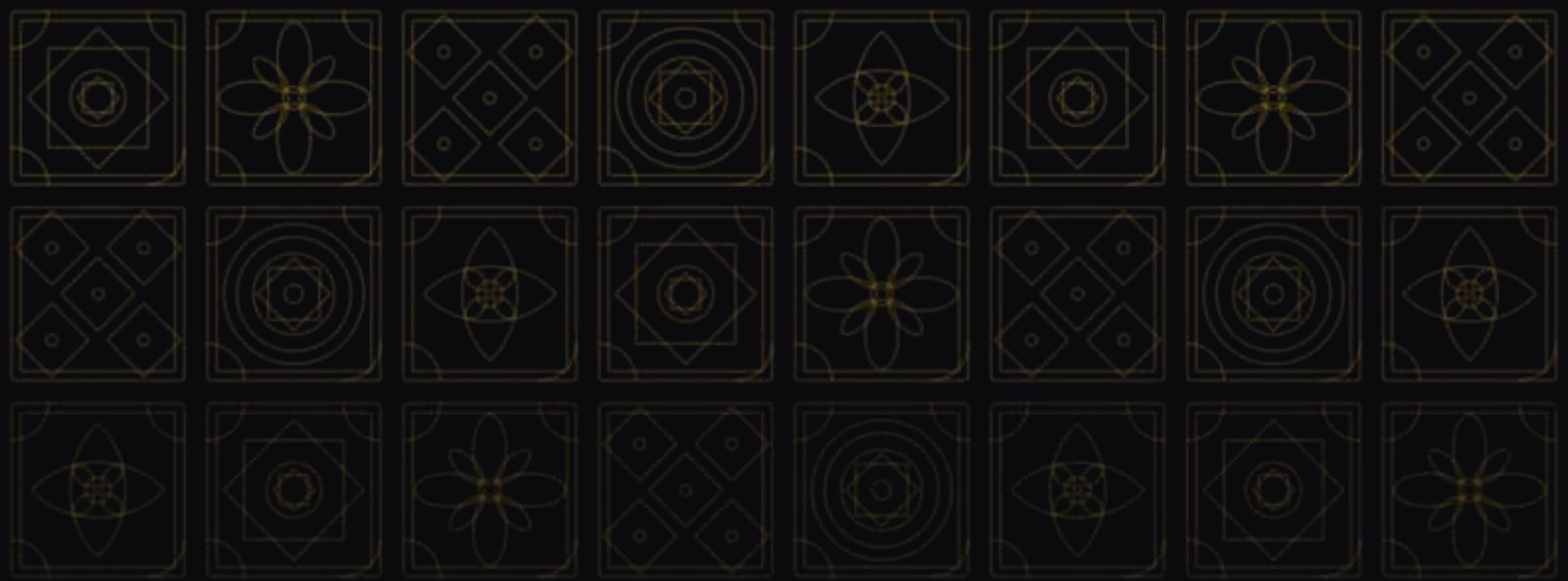




PROGRAMME DE FORMATION

Fondamentaux de la conception de systèmes

Concevoir des systèmes évolutifs et fiables, et en expliquer les compromis

21 HEURES

POUR Développeurs et ingénieurs qui conçoivent ou relisent des systèmes logiciels



Mohamed Aziz BEN HAH

mohamedazizbenhaha.netlify.app

Présentation

Cet atelier apprend aux participants à concevoir des systèmes logiciels qui montent en charge et restent fiables, et à expliquer leurs choix. Ils abordent les fondamentaux de l'évolutivité, de la latence et de la cohérence, les styles d'API et l'authentification, le cache et les CDN, le stockage des données, la réplication et le partitionnement, la messagerie asynchrone, et les styles d'architecture avec les motifs qui les rendent résilients et observables. Des études de cas classiques rassemblent le tout, et l'atelier s'achève par un projet final où chaque équipe conçoit un système pour un cahier des charges et le défend lors d'une revue par les pairs.

Vue d'ensemble

#	PHASE	THÈMES	HEURES
01	Fondamentaux, API et cache	Évolutivité, CAP et cohérence, styles d'API, authentification, cache, CDN	6 h
02	Données, messagerie et architecture	SQL et NoSQL, réplication, partitionnement, files et Kafka, styles d'architecture, résilience	6 h
03	Études de cas	Raccourcisseur d'URL, messagerie instantanée, système de paiement, fil d'actualité	3 h
04	Projet final	Un système conçu pour un cahier des charges, défendu en revue par les pairs	3 h
	Réserve	Rattrapage, révision, TP supplémentaires	3 h
	Total		21 h

Informations clés

PUBLIC	Développeurs et ingénieurs qui conçoivent ou relisent des systèmes logiciels
PRÉREQUIS	Programmation dans un langage (cours : Fondamentaux de la programmation Python), ou équivalent, et notions de bases de données et de HTTP
DURÉE	21 h : 18 h de chapitres + 3 h de réserve
FORMAT	Atelier : cours courts, chacun suivi d'exercices de conception en petits groupes ; études de cas et revue par les pairs
ENVIRONNEMENT DE TP	Un portable par participant avec un navigateur et un outil de schémas (Excalidraw ou draw.io) ; petites démonstrations avec Redis et Kafka sous Docker menées par le formateur ; ni compte cloud ni coût
ÉVALUATION	Exercices de conception revus dans chaque phase et une conception finale défendue en revue par les pairs ; attestation de formation pour les participants qui valident les deux
RÉSULTAT	Les participants savent transformer des exigences en conception de système avec API, modèle de données, cache, messagerie et plan de montée en charge, et en défendre les compromis

PHASE 01

6 H

Fondamentaux, API et cache

Les participants apprennent le vocabulaire de la conception de systèmes : évolutivité, latence et débit, théorème CAP et modèles de cohérence. Ils choisissent ensuite comment les services communiquent et s'authentifient, et où le cache et les CDN réduisent latence et charge. Exercice validé : une estimation de capacité rapide et une API avec stratégie de cache pour un service de partage de photos.

#	CHAPITRE	CONTENU	FORMAT	HEURES
1.1	Fondamentaux	Évolutivité, latence et débit, théorème CAP, modèles de cohérence, estimations de capacité	Cours + atelier	3 h
1.2	API et communication	REST, GraphQL, gRPC, webhooks, WebSockets, passerelles d'API, authentification (OAuth 2.0, OpenID Connect)	Cours + atelier	1,5 h
1.3	Cache et CDN	Stratégies de cache, éviction, Redis, CDN	Cours + atelier	1,5 h

PHASE 02

6 H

Données, messagerie et architecture

Les participants choisissent et font monter en charge les bases de données avec la réplication, le partitionnement et les index, découplent les services avec files d'attente, publication/abonnement et Kafka, et comparent les styles d'architecture avec les motifs qui les rendent fiables : limitation de débit, nouvelles tentatives, disjoncteurs, observabilité et objectifs de niveau de service. Exercice validé : le modèle de données, la clé de partitionnement et le flux de traitement des commandes d'une boutique en ligne, avec la gestion des pannes.

#	CHAPITRE	CONTENU	FORMAT	HEURES
2.1	Données	SQL et NoSQL, réplication, partitionnement (sharding), indexation	Cours + atelier	3 h
2.2	Systèmes asynchrones	Files d'attente, publication/abonnement, Kafka, idempotence, sagas	Cours + atelier	1,5 h
2.3	Styles d'architecture	Monolithe, microservices, architecture événementielle, serverless ; limitation de débit, nouvelles tentatives, disjoncteurs, observabilité, SLO	Cours + atelier	1,5 h

PHASE 03

3 H

Études de cas

Les participants appliquent toute la méthode à des problèmes classiques, des exigences et estimations jusqu'à l'architecture finale et ses points faibles, en comparant leurs conceptions entre eux. Exercice validé : une étude de cas conçue par chaque groupe et critiquée par un autre.

#	CHAPITRE	CONTENU	FORMAT	HEURES
3.1	Études de cas	Raccourcisseur d'URL, application de messagerie instantanée, système de paiement, fil d'actualité	Cours + atelier	3 h

PHASE 04

3 H

Projet final

Par équipes de deux ou trois, les participants conçoivent un système pour un cahier des charges écrit, par exemple une application de VTC, en réutilisant les méthodes de l'atelier, puis le défendent devant une autre équipe.

#	CHAPITRE	CONTENU	FORMAT	HEURES
4.1	Cahier des charges et conception	Exigences et estimations, schéma d'architecture, modèle de données, plan de montée en charge	Projet	1,5 h
4.2	Revue par les pairs	Présenter la conception et ses compromis, relire la conception d'une autre équipe ; retours	Projet	1,5 h

PROJET FINAL

Conception d'un système sur cahier des charges

Un système conçu à partir d'un cahier des charges écrit, par exemple une application de VTC : exigences et estimations de capacité, schéma d'architecture, API, modèle de données, cache et messagerie, plan de montée en charge et compromis derrière chaque choix.

OUTILS Schémas d'architecture (Excalidraw ou draw.io), estimations de capacité, document de conception

LIVRABLE Document de conception avec schémas, modèle de données et plan de montée en charge, notes de revue par les pairs